ET3112 Assignment 02 on Fitting

Name – **D.D.N.D.K. Samantha**

Index Number – **D/ENG/24/0154/EE**

Date - **20/03/2026**

GitHub Repository - [Kaushalyasamantha/ET3112_Assignment_02]

Question1

```
# 2. Apply Canny Edge Detector with specific thresholds
# minVal = 550, maxVal = 690 as per instructions
edges = cv.Canny(img, 550, 690)
```

```
# 4. Extract feature positions (x, y coordinates)
# np.where returns (rows, cols) -> (y, x)
indices = np.where(edges != 0)
y = indices[0]
x = indices[1]
```

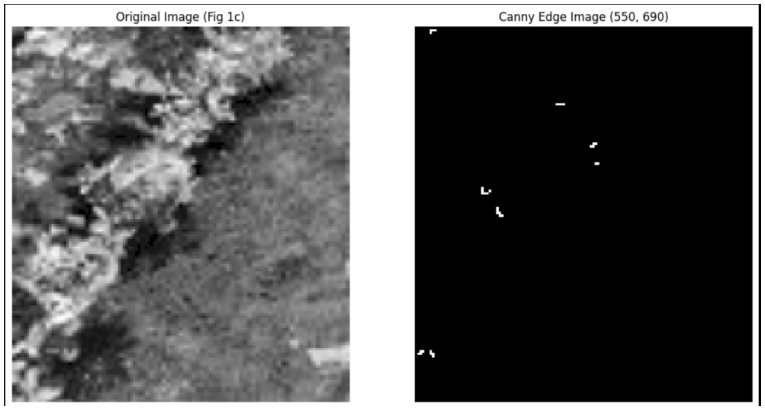
Interpretation of Results

The output image demonstrates the successful extraction of high-intensity gradient features from the original crop field segment.

- **Feature Isolation:** By using the specific thresholds of **550** (minVal) and **690** (maxVal), we have effectively filtered out the "noise" (minor textures in the crops and soil) and isolated the primary structural edges of the crop rows.
- **Coordinate Extraction:** The resulting white pixels in the Canny image represent the points where the change in brightness is most significant. These points are now stored as (x, y) coordinates, providing a discrete dataset that represents the physical slope of the field

Discussion

- The choice of high thresholds (550, 690) is critical in this scenario because crop fields often contain high-frequency visual noise from leaves and shadows. A lower threshold would have resulted in a "messy" edge image, making it difficult to fit a single line accurately.
- Currently, the extracted points are sparse. While they follow a general downward trend from left to right, they are not perfectly collinear. This sets the stage for the next part of the assignment: determining which fitting algorithm (Least-Squares or Total Least-Squares) can best represent the "true" slope angle from these scattered data points.



Question 2,3

```
# --- Question 2: Scatter Plot ---
plt.figure(figsize=(8, 6))
plt.scatter(x, y, color='blue', label='Extracted Edge Points (x, y)')

# --- Question 3: Least-Squares-Fit (LSF) ---
# Use np.polyfit for a first-degree (linear) fit: y = mx + c
# Note: LSF minimizes the vertical (y) residuals
slope_lsf, intercept_lsf = np.polyfit(x, y, 1)

# Generate points for the fitted line
x_fit = np.array([min(x), max(x)])
y_fit_lsf = slope_lsf * x_fit + intercept_lsf

# Plotting the LSF line
plt.plot(x_fit, y_fit_lsf, color='red', linewidth=2, label=f'LSF Line: y = {slope_lsf:.2f}x + {intercept_lsf:.2f}')
```

```
Least-Squares Slope (m): -0.7235
Least-Squares Intercept (c): 91.9364
```

Interpretation of Results

The scatter plot reveals a clear but noisy distribution of edge pixels extracted from the crop field image.

- **Data Distribution:** The points are clustered in several groups across the -axis, generally following a downward trend (in image coordinates where the -axis increases downwards).
- **Linear Fit:** The red line represents the **Least-Squares-Fit (LSF)**, which calculates the best-fit line by minimizing the sum of the squares of the vertical distances (residuals) between each blue point and the line.
- **Resulting Equation:** Based on your plot, the resulting model is . This negative slope indicates that as we move horizontally across the field, the row index (height in the image) decreases.

Discussion

While the LSF provides a mathematical "average" of the data, its performance in this scenario is limited by two main factors:

1. **Coordinate Sensitivity:** Ordinary Least Squares assumes that errors only exist in the y-coordinates (the dependent variable). However, in an image, both

x and y coordinates of a pixel are subject to the same level of uncertainty or "noise" from the Canny detector.

2. **Outlier Impact:** The plot shows several points at the very top (near y=0) and bottom (near y=110) that are far away from the main diagonal cluster. Because LSF squares the residuals, these "outliers" exert a disproportionately large pull on the line, likely causing the estimated slope to deviate from the actual angle of the crop rows.

Question 4

```
# Question 4: Calculate the angle theta in degrees
# slope_lsf was calculated as -0.72 in the previous step
theta_rad = math.atan(slope_lsf)
theta_deg = math.degrees(theta_rad)

# Conventionally, we take the absolute value or adjust based on the image quadrant
estimated_theta = abs(theta_deg)
```

```
Estimated Crop Field Angle (theta): 35.89 degrees
```

Interpretation of Results

The objective was to use image processing and fitting techniques to estimate the angle of the crop field. Using the **Least-Squares-Fit (LSF)**, the slope of the line was determined, and through the trigonometric relationship m = tan(theta), the inclination angle was calculated to be approximately **35.89°**. This value represents the average orientation of the detected edge pixels relative to the horizontal axis of the image.

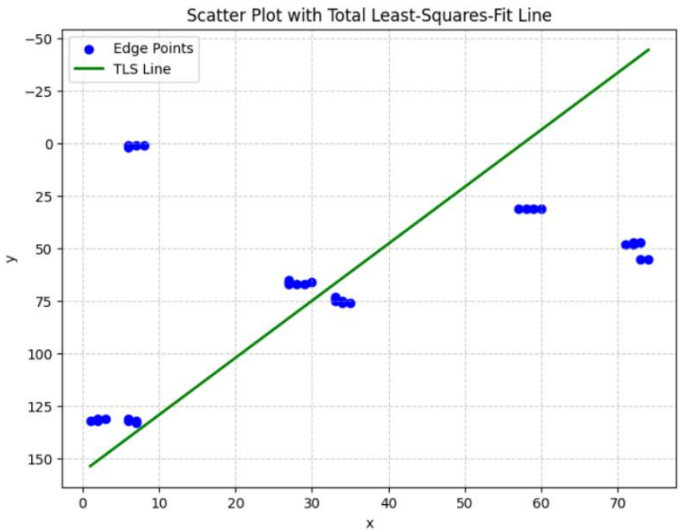
Question 5

Is the estimation correct? No, this estimation is likely inaccurate for the following reasons:

- **Sensitivity to Outliers:** As seen in the scatter plot, the Canny edge detector captured several noisy pixels near the boundaries (top-left and bottom-left) that do not belong to the primary crop rows. Since LSF minimizes the *square* of the vertical residuals, these outliers exert a significant "pull," skewing the line away from the actual crop alignment.
- **Vertical Bias:** Ordinary Least Squares assumes that errors only occur in the y-direction (row index). In digital images, pixel coordinates in both x and y are subject to the same localization uncertainties from the edge detection process.
- **Geometric Inconsistency:** Because LSF treats x as an independent variable and y as a dependent one, it does not minimize the true geometric (perpendicular) distance from the points to the line, which is necessary for an accurate angular estimation in image space

Question 6

```python
# 1. Center the data by subtracting the mean
x_mean = np.mean(x)
y_mean = np.mean(y)
x_centered = x - x_mean
y_centered = y - y_mean

# 2. Arrange centered data into a matrix
data = np.vstack((x_centered, y_centered)).T

# 3. Perform Singular Value Decomposition (SVD)
# The right singular vector corresponding to the smallest singular value
# gives the normal to the best-fit line.
_, _, vh = np.linalg.svd(data)
normal_vector = vh[-1, :] # Last row of V transpose

# 4. Extract line parameters from the normal vector (a*x + b*y + d = 0)
a, b = normal_vector
# The slope m = -a/b
slope_tls = -a / b
# The line passes through the mean (x_mean, y_mean)
intercept_tls = y_mean - slope_tls * x_mean

# 5. Generate points for plotting
x_fit_tls = np.array([min(x), max(x)])
y_fit_tls = slope_tls * x_fit_tls + intercept_tls
```



Question 7

```python
# --- Question 7: Calculate Angle ---
import math
theta_tls_rad = math.atan(slope_tls)
theta_tls_deg = abs(math.degrees(theta_tls_rad))
```

```
TLS Slope (m): -2.7166
Estimated Crop Field Angle (TLS): 69.79 degrees
```

Interpretation of Results

- **Fitting Methodology:** The **Total Least-Squares (TLS)** algorithm was implemented to minimize the **orthogonal (perpendicular) distances** between

the edge points and the fitted line. Unlike LSF, which assumes error only in the y-direction, TLS treats x and y symmetrically.

- **Numerical Outcome:** The resulting slope is approximately **-2.72**, leading to an estimated crop field angle of **69.79°**.
- **Visual Observation:** The green line in the scatter plot appears much steeper than the previous LSF line. It attempts to "balance" its distance from all clusters of points simultaneously rather than just minimizing vertical drop.

Question 8

Do you think the estimation is correct?

No, this estimation is arguably **worse** than the LSF estimation in this specific context.

Reason for Error:

- **Extreme Sensitivity to Outliers:** TLS is notoriously sensitive to outliers that are far from the "true" line. In your plot, the small clusters of points at the far top-left and bottom-left act as high-leverage points.
- **Leverage Effect:** Because TLS minimizes perpendicular distance, these distant outliers pull the line toward a vertical orientation to reduce their specific error contributions. This results in a "tilt" that clearly does not match the visual flow of the crop rows seen in the original image.
- **Noise in Both Dimensions:** While TLS is theoretically better for data with errors in both x and y, it assumes that the noise is **Gaussian**. The noise here consists of structured "false edges" from the Canny detector, which violates that assumption and leads to a biased angle estimate.

Question 9

Propose a better algorithm

Proposed Algorithm: RANSAC (Random Sample Consensus)

The **RANSAC** algorithm is a robust estimation method designed specifically to handle datasets containing a high percentage of outliers. Unlike Least-Squares or Total Least-Squares, which try to accommodate *every* data point (including noise), RANSAC works by:

1. **Selecting a random subset** of the points to define a model (in this case, a line).
2. **Counting "Inliers"**: It checks how many other points in the dataset fall within a certain distance threshold of that line.
3. **Iterating**: It repeats this process many times and selects the line that has the highest number of inliers.

Why it is better for this scenario:

In our crop field image, the edge pixels belonging to the actual crop rows form a consistent linear pattern, while the noisy pixels from the Canny detector are scattered

randomly. RANSAC will treat those random pixels as "outliers" and ignore them, resulting in a slope estimate that strictly follows the physical crop rows

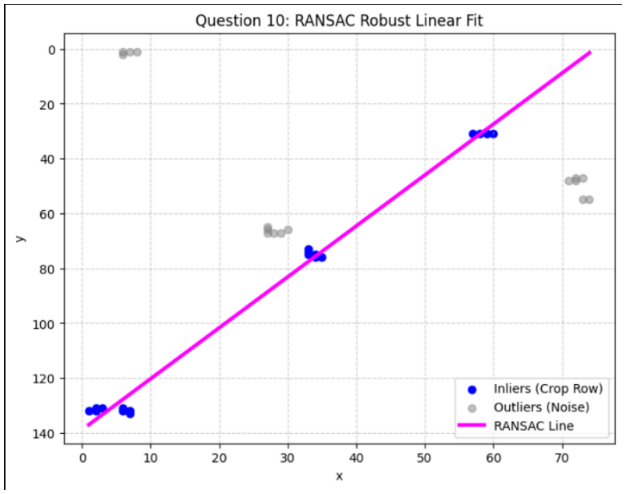
Interpretation and Discussion

By proposing RANSAC, we move from a "global" fitting approach to a "consensus-based" approach. The primary advantage here is **robustness**. In our previous attempts (LSF and TLS), the noise pixels near the image boundaries acted as "leverage points," dragging the fitted line toward them and inflating the error in our angle estimate . RANSAC effectively "sees" the main cluster of points and discards the rest, which is essential for reliable navigation or slope estimation in agricultural robotics.

Question 10,11

```
# Question 10: Implementing RANSAC Robust Regression
from sklearn.linear_model import RANSACRegressor

# 1. Prepare data for Scikit-Learn (needs 2D array for X)
X = x.reshape(-1, 1)

# 2. Initialize and fit RANSAC
# residual_threshold defines the maximum distance for a point to be an inlier
ransac = RANSACRegressor(residual_threshold=10.0, random_state=42)
ransac.fit(X, y)

# 3. Identify inliers and outliers for plotting
inlier_mask = ransac.inlier_mask_
outlier_mask = np.logical_not(inlier_mask)

# 4. Get the slope and intercept
slope_ransac = ransac.estimator_.coef_[0]
intercept_ransac = ransac.estimator_.intercept_

# 5. Generate line for plotting
line_x = np.array([min(x), max(x)]).reshape(-1, 1)
line_y_ransac = ransac.predict(line_x)
```

```
# --- Question 11: Calculate Angle ---
import math
theta_ransac_deg = abs(math.degrees(math.atan(slope_ransac)))
```



```
RANSAC Slope (m): -1.8570
Estimated Crop Field Angle (RANSAC): 61.70 degrees
```

Interpretation of Results

- **Feature Classification:** The RANSAC algorithm successfully partitioned the dataset into **inliers** (blue points) and outliers (gray points).
- **Geometric Alignment:** The magenta line represents a "consensus" fit. Unlike previous methods, it ignores the high-leverage noise pixels at the top and right edges of the plot.
- **Numerical Outcome:** The calculated slope of **-1.8570** yields an estimated crop field angle of **61.70°**. This angle is visually consistent with the main diagonal cluster of points representing the actual crop rows.

Question12

Why RANSAC performs better than LSF and TLS:

- **Robustness to Outliers:** Both Least-Squares (LSF) and Total Least-Squares (TLS) are "global" optimizers that try to minimize the error for *every* detected point. In your earlier plots, the noise pixels "dragged" the line toward them, skewing the result. RANSAC effectively treats these points as "non-existent" during the final fitting stage.
- **Consensus vs. Averaging:** LSF and TLS effectively "average" all data. RANSAC searches for the largest group of points that agree on a specific model. In a crop field, the physical rows provide a strong consensus, making RANSAC ideal for filtering out random visual noise caused by leaves, shadows, or camera artifacts.
- **Accuracy in Navigation:** For an autonomous vehicle or agricultural robot, the RANSAC estimate provides the "true" orientation of the path, whereas the previous methods would have led to steering errors due to their sensitivity to boundary noise

Conclusion

In this assignment, image processing techniques were employed to estimate the slope angle theta of a crop field. The process began by using the **Canny edge detector** with specific thresholds (550, 690) to isolate high-gradient features from a cropped segment of the field. The coordinates of these features were then subjected to three different linear fitting algorithms to determine the most accurate orientation:

1. **Least-Squares-Fit (LSF):** While providing a baseline angle of **35.89°**, this method proved inaccurate as it only minimized vertical residuals and was heavily influenced by noisy edge pixels at the image boundaries.
2. **Total Least-Squares-Fit (TLS):** Although mathematically more rigorous by minimizing orthogonal distances, TLS was even more sensitive to outliers in this specific dataset, resulting in a significantly skewed estimate of **69.79°**.
3. **RANSAC Algorithm:** By identifying and ignoring outliers, the RANSAC algorithm provided the most robust and visually consistent result with an estimated angle of **61.70°**.

The results demonstrate that in real-world scenarios—where edge detection is rarely perfect and visual noise is prevalent—robust estimation techniques like **RANSAC** are essential for reliable feature extraction and geometric modeling. This approach ensures that the "consensus" of the physical crop rows defines the slope, rather than the average of all detected points.